S2 E16

BK

Season 2: Retrieval and RAG

Season 2 recap.

One RAG system, built a stage at a time and measured at every step. From 243 right answers to 354, and from 51 leaks to none. What joined, what did not, and what is left.

Brendan Kelly

Applied AI

The season in four numbers

The same 520 questions, at the start of the season and at the end.

RIGHT ANSWERS

243 → 354

of 400 answerable questions

WRONG ANSWERS

157 → 46

of 400 answerable questions

LEAKS

51 → 0

public questions shown staff-only text, of 480

SAID “NOT FOUND”

53 → 107

of 120 the library cannot answer

The start is E4’s rerun of the first system on the season’s final models, so the comparison is fair. On E2’s original, pricier models it scored 217 right.

What the season set out to do

Build one RAG system for a fictional appliance firm, Halvard Home, and measure every change to it.

THE LIBRARY

344

documents

304 public, current

32 withdrawn

4 superseded

4 staff-only

THE TEST

520

questions

400 the library can answer

120 it cannot

THE METHOD

1

stage changed at a time

a bar set before each run

joins only if it clearly beats the system

Why this way

Change one stage, keep everything else fixed, and any difference in the results belongs to that stage.

The system

The system the season built

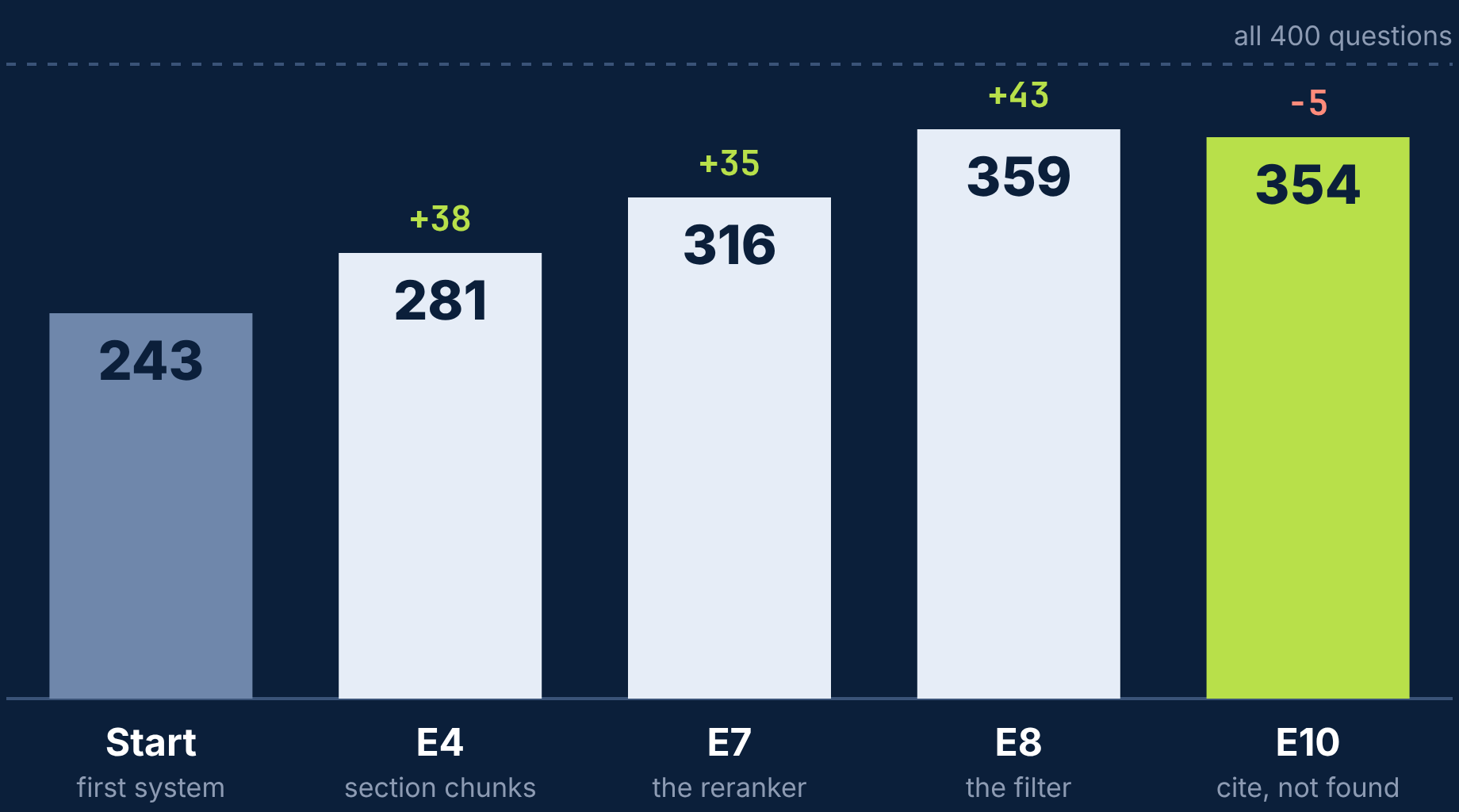
All eleven stages, as they stand now. Lime: changed this season. Coral: tested, kept as it was.

1	**Load** OFFLINE	344 documents, each labelled with access and status	E2
2	**Chunk** OFFLINE	one chunk per section, not fixed windows	E4
3	**Embed** BOTH	text-embedding-3-small; a bigger model lost	E2
4	**Rewrite** LIVE	none: the question as typed	E2
5	**Search** LIVE	vector search, top 50	E2
6	**Filter** LIVE	permission and version, before ranking	E8
7	**Rerank** LIVE	gpt-6-luna reorders the top 20	E7
8	**Prompt** LIVE	top 5; answer only from them, cite, or say not found	E10
9	**Answer** LIVE	gpt-6-luna, reasoning off	E4
10	**Cite** LIVE	each [P1] marker checked in code	E10
11	**Score** TESTS	gemini-3.6-flash judge, checked by hand	E3

The climb

Right answers climbed in four steps

Right answers of 400, after each change that joined the system.



Read the last step

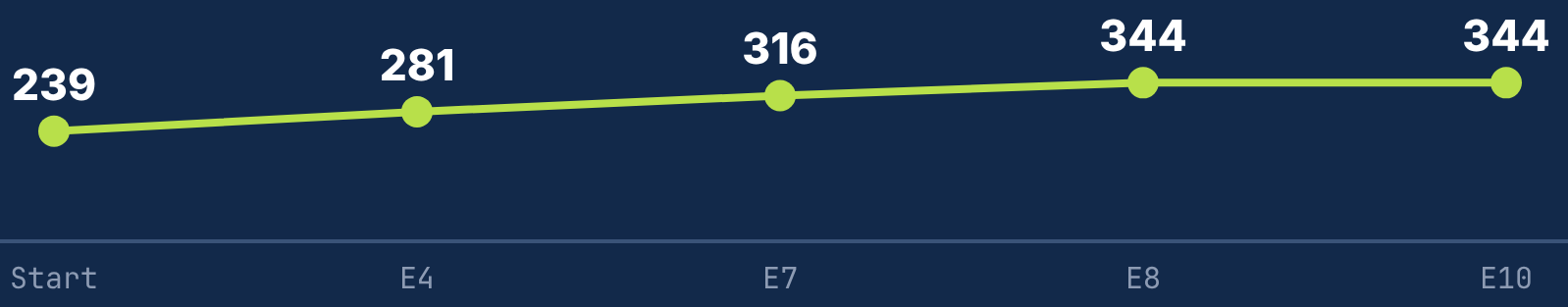
E10 cost 5 right answers, within luck, and bought something else: every answer now cites its source, and says "not found" far more often.

Search, “not found” and leaks, step by step

Not every number moved the right way at every step.

Found the right passage of 400

Rose at every step that touched finding: chunks, the reranker, the filter.



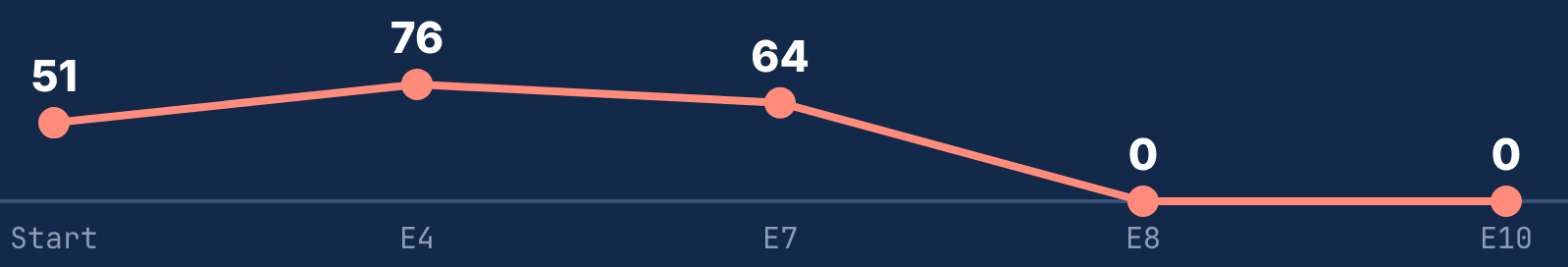
Said “not found” of 120

Fell at first, then doubled once wrong passages were filtered out and the model was told it may decline.



Leaks of 480

Section chunks raised leaks; the E8 filter ended them.



The season

Sixteen episodes, one system

Lime: joined the system. Coral: tested, did not join. Grey: taught or traced.

FOUNDATIONS

E0	Season intro	intro
E1	Why retrieval exists	the case for RAG
E2	Your first RAG system	the starting point
E3	Measuring it	how we score

FINDING THE RIGHT PASSAGE

E4	Chunking	joined, +38
E5	Search	not joined
E6	Rewriting the question	not joined
E7	Reranking	joined, +35
E8	Filters	joined, +43

USING THE PASSAGE

E9	Building the prompt	not joined
E10	Grounding and citations	joined
E11	Field note: citations	tested E9, E10

RUNNING IT AND DECIDING

E12	Field note: embeddings	not joined
E13	Field note: long context	tested S1, E1
E14	Tracing a wrong answer	every error traced
E15	When RAG is wrong	four alternatives
E16	Season 2 recap	you are here

What joined

The four changes that joined

Each cleared a bar written down before its run.

E4 · STAGE 2

Cut along each document’s sections

243 → 281

right answers of 400

Answers no longer split across two chunks.

E7 · STAGE 7

Let a model reorder the top 20

281 → 316

right answers of 400

Found passages ranked too low moved into the top 5.

E8 · STAGE 6

Filter by permission and version

316 → 359

right answers of 400

Leaks fell from 64 to 0.

E10 · STAGE 8 & 10

Answer only from the passages, and cite

83 → 107

said “not found”, of 120

Every answer now points to its source.

What joined

The biggest gain came from a label, not a model

Two changes, side by side. One read two labels on each document. The other cost 6.5 times as much per token.

E8: TWO LABELS PER DOCUMENT

+43

right answers, and leaks from 64 to 0. No model call at all.

E12: A BIGGER EMBEDDING MODEL

−17

right answers, at 6.5 times the price per token.

The lesson for leaders

The season’s gains came from structure: how documents are cut, labelled and ordered. Not one came from a bigger model.

What did not

Four changes that did not join, and why

Three of them helped something real. The bar was strict on purpose.

E5 · SEARCH

Add keyword search

HELPED 19 more right answers: 299 against 280.

BUT found the right passage less often: 276 against 281.

E6 · REWRITE

Rewrite follow-up questions

HELPED follow-ups found rose from 7 to 34 of 40.

BUT 40 questions could not lift 400 clear of luck.

E9 · PROMPT

Allow “not found”

HELPED correct declines rose from 83 to 105 of 120.

BUT spread over 520 questions, it overlapped. It joined later, inside E10.

E12 · EMBED

Use a bigger embedding model

HELPED nothing: it found less, on our questions.

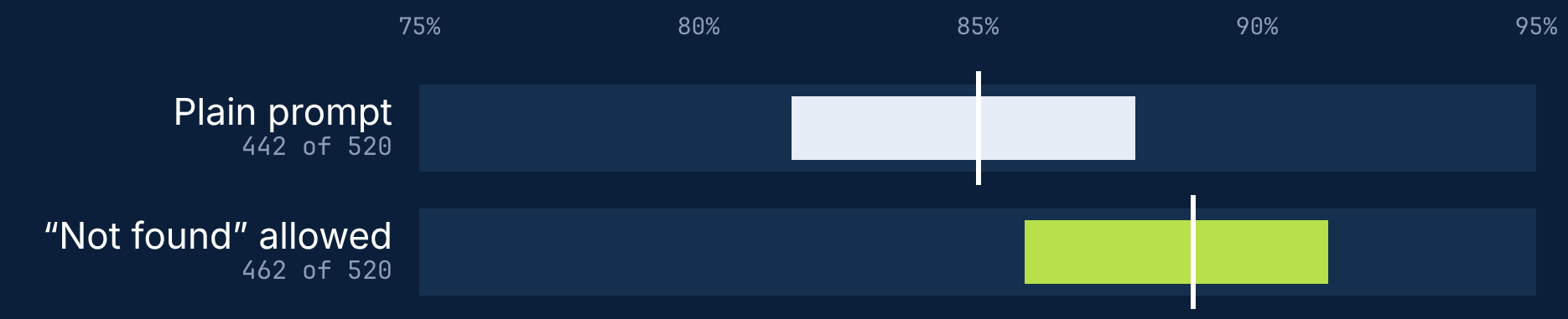
BUT 17 fewer right answers, at 6.5 times the price.

What did not

Why a strict bar is a feature

Every score has a likely range, because luck moves it a few points. A change joins only if its range sits clear above the system’s.

E9: HANDLED RIGHT, OF 520



What it buys	Nothing joins on luck. Every change in the final system is real.
What it costs	Some real gains wait outside, like E5 and E6. They are the next things to try.

One question

One question, followed through the season

A PART-CODE QUESTION FROM THE TEST

What does error E-2194 mean? Answer: an interrupted factory reset on the Ashby tumble dryer

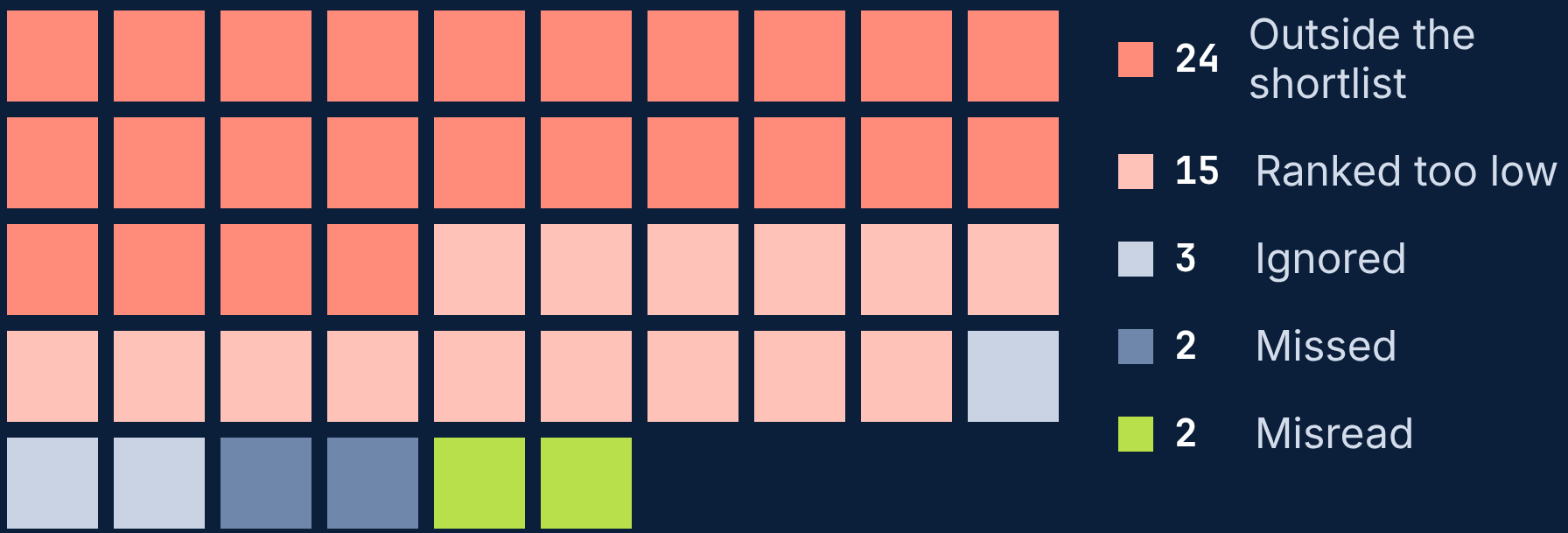
E5	Vector search ranks the right article 10th, behind other “error E-something” articles.	10th
E6	Rewriting adds generic words, and pushes it down to 17th.	17th
E7	The reranker reads the top 20, spots “E-2194”, and puts it first.	1st
E10	The answer cites its source: “…so your previous preferences may still be active. [P1]”	cited
E12	A bigger embedding model ranks it 1st straight away, yet loses 17 answers overall.	1st

The lesson

One question tells a story. Only the whole test set tells you whether to change anything.

What is left: 46 wrong answers

Each square is one wrong answer, coloured by the first stage that lost the right passage (E14).



Mostly ranking	39 of 46. Most never reached the reranker’s top 20.
Mostly two kinds	Follow-ups (19) and part codes (14): the targets of E6 and E5.

Surprises

Six results that went against expectation

Each was written down as a prediction first. That is how a surprise becomes visible.

51 → 76

Better chunks raised leaks, until the filter came.

`E4 chunking`

64 → 126

Filtering old versions alone doubled leaks.

`E8 filters`

−17

A bigger embedding model gave fewer right answers.

`E12 embeddings`

359 = 359

Moving the best passages to the ends changed nothing.

`E9 prompt`

75 vs 71

Sending the whole library was as accurate, and no slower.

`E13 long context`

+2 tokens

Requiring citations changed nothing else.

`E11 citations`

The cost

What it costs to run, and to test

Leaders decide on cost. Here are the season’s numbers.

$0.0016

per question to answer, with the reranker and the answer model

E13

$157

a month at 100,000 questions; sending the whole library instead: $1,606

E13

<$20

for every experiment from E4 to E13 together, on the cheapest models

E4 to E13

Reused runs	E11, E14 and E15 cost nothing: they read runs already paid for.
The lesson	Testing a change costs a few dollars. Shipping an untested one costs wrong answers.

E2 and E3 ran on pricier models and are not included; nor are $6.50 of runs discarded in E4.

Honest notes

What changed from the plan, stated plainly

Three choices differ from what the season set out to do. Each was made for cost or speed, and tested.

Cheaper models	From E4, gpt-6-luna answered and gemini-3.6-flash judged, in place of pricier models. E4 reran the starting system on them, so every comparison is fair.
An LLM reranker	The planned local cross-encoder took 62 seconds a question on a CPU. An LLM was as accurate, at about 1 second (E7).
The judge check	A second model marked all 50 sample answers, plus 10 marked by hand, not 50 by hand (E3).

Every decision rule: building the system

One rule per episode, from E1 to E8.

E1	Use retrieval when the answer lives in your own data, then log what it retrieved and measure search and answer separately.
E2	Build the simplest complete pipeline and measure it before tuning any stage.
E3	Score search and answer separately, include questions your documents cannot answer, and check your judge before you trust it.
E4	Cut documents along their own structure, then test it.
E5	Test search on each kind of question, keep keyword search for exact codes, and merge rankings only where tests show it helps.
E6	Rewrite a question only where tests show that kind fails, and weigh the extra call against what it recovers.
E7	Add a reranker when the right passage is found but ranked too low, and measure its cost per question.
E8	Filter by permission and version before ranking, never after, and test that a restricted passage cannot reach the prompt.

Every decision rule: running it and deciding

One rule per episode, from E9 to E15.

E9	Send the fewest passages that keep the answer score, and let the model say “not found”.
E10	Require every claim to cite a passage, then check that the passage supports it.
E11	Asking for citations costs nothing in accuracy here: require them, and check them.
E12	Swap embedding models in one step, and compare answers question by question.
E13	Use long context when the documents fit and change rarely; otherwise, retrieval.
E14	When an answer is wrong, find the first stage that lost the right passage before changing anything.
E15	Use RAG when the answer is a passage; use something else for a calculation, a behaviour, or a live state.

Lessons

Five lessons to take into any RAG project

If you remember nothing else from the season.

1 **Measure search and answers apart**
A wrong answer is either a passage not found or a passage misread. Fix the one that failed.

2 **Write the bar down before the run**
Then the result cannot bend it, and a surprise becomes visible.

3 **Structure beats size**
Chunks, labels and order gave every gain. A bigger model gave none.

4 **Trace before you fix**
Find the first stage that lost the answer. Sometimes it is the documents.

5 **Choose by the shape of the answer**
RAG is for passages. Records, behaviour and live state need other tools.

For leaders

Three questions for any team building RAG

The season, compressed into what to ask on Monday.

Ask your team

Do we measure finding and answering separately, on our own questions?

Ask your team

Is the bar for a change written down before we test it?

Ask your team

When an answer is wrong, can we say which stage caused it?

What is next

What this season left for later

Four topics touched here, each with a season of its own.

SEASON 4 · ADAPTING

Fine-tuning

Changing how a model behaves, not what it knows.

SEASON 5 · AGENTS

Tools and agentic retrieval

Live systems, and models that decide what to search for.

SEASON 7 · DATA

Document parsing

Getting clean text and labels out of real documents.

SEASON 11 · SECURITY

Prompt injection

When a retrieved document carries instructions of its own.

S2 E16

For engineers

Run the whole season yourself

Every result is committed. Rebuilding them costs nothing.

```
# the season recap chart and numbers: no key, no cost
$ uv run python season2/recap.py

# any episode's results, for example E8
$ uv run lab rag analyse episodes/s2-e8-filters/config.yaml
```

The final system	`season2/reference.yaml`: every stage's setting.
What joined, when	`season2/reference_history.yaml` and `scoreboard.json`.

The season in one rule

Build the whole pipeline first, measure each stage on your own questions, and change one stage at a time against a bar set before the run.

Next: Season 3, AI strategy

The leadership season. Where AI earns its place, which use case goes first, and choices that outlast the next model release.

Where this fits

The series map

S1 LLMs	**S2** Retrieval	**S3** Strategy
S4 Adapting	**S5** Agents	**S6** Economics
S7 Data	**S8** Evaluation	**S9** AI teams
S10 Reliability	**S11** Security	**S12** Delivery
S13 Performance	**S14** Resilience	**S15** Governance

☐ Leadership season

Follow linkedin.com/in/brendan-kelly-irl

Episode guide brendankellyai.github.io

Code github.com/BrendanKellyAI/applied-ai-lab